Quantum-Accelerated Sequence Similarity Comparison and Sketch Construction

Zhezheng Xander Song* Carl Kingsford†

November 5, 2026

Abstract

Sketch-based methods such as MinHash enable fast estimation of sequence similarity, but their cost still grows at least linearly with the number of sequences, and becomes substantial in tasks that require many comparisons. We study how quantum algorithms can accelerate three stages of sequence comparison under a QRAM model: pairwise similarity estimation, database search, and sketch construction. We first show that, after near-linear-time preprocessing, quantum amplitude estimation can estimate the containment index and Jaccard index of two k-mer sets to additive error ε in $\widetilde{O}(1/\varepsilon)$ time, compared with $O(1/\varepsilon^2)$ for classical sampling. Building on this estimator, we develop error-tolerant quantum search algorithms over a preprocessed database of R references, which find the best-matching reference for a query and the most similar pair of references, up to additive error ε, in $\widetilde{O}(\sqrt{R}/\varepsilon)$ and $\widetilde{O}(R/\varepsilon)$ expected time, compared with $\widetilde{O}(R/\varepsilon^2)$ and $\widetilde{O}(R^2/\varepsilon^2)$ for classical sampling. Finally, we use quantum minimum finding to construct MinHash signatures for s prescribed hash functions of a sequence of length L with n distinct k-mers in $\widetilde{O}(L+s\sqrt{n})$ expected time including preprocessing, compared with $\widetilde{O}(L+ns)$ for direct classical construction. We implement the core procedures in Q# and test them in small-scale simulation. Our results show that quantum speedups are especially promising for large-scale sequence comparison, where the cost of preparing an input representation can be amortized across many estimates, searches, or hash functions.

*Ray and Stephanie Lane Computational Biology Department, Carnegie Mellon University, 5000 Forbes Ave., Pittsburgh, PA 15213, USA. Email: xanders@cs.cmu.edu.

†Ray and Stephanie Lane Computational Biology Department, Carnegie Mellon University, 5000 Forbes Ave., Pittsburgh, PA 15213, USA. Email: carlk@cs.cmu.edu. This work was supported in part by the US National Science Foundation [III-2232121], the US National Institutes of Health [R01HG012470].

1 Introduction

Sequence similarity estimation is a fundamental problem in genomic analysis, with applications in metagenomic profiling [15, 30], species identification [8, 16], genome clustering [23], contamination screening [1], and large-scale database search [5]. Sketching makes these analyses scalable by providing compact sequence representations that enable fast similarity estimation [19]. An example is MinHash [7], which estimates the Jaccard index between sets using small sketches and has been successfully adopted in bioinformatics tools like Mash [25] for rapid genome and metagenomic analysis. Mash Screen [24] extends this approach to containment estimation, determining which references in a sketch database are present in a query sequence, such as a metagenome. A family of MinHash variants has since been developed for different settings: Ordered MinHash [18] preserves the relative order of k-mers to approximate edit distance rather than Jaccard index, while FracMinHash [15] retains a fixed fraction of hash values so that sketch size scales with sequence length, enabling accurate containment estimation between sets of very different sizes. Despite their substantial computational advantages, classical sketching methods can still face significant computational bottlenecks as genomic analyses scale along several dimensions: the number of sequences can grow rapidly in large genome or metagenomic databases such as GTDB [27] and NCBI RefSeq [22], individual sequences can become much longer when analyzing whole genomes or long-read data, and the number of required similarity computations can increase rapidly in tasks such as all-versus-all comparison [16] and database search [8].

Quantum computing provides a fundamentally different computational paradigm based on superposition, interference, and measurement [21]. Quantum algorithms can achieve asymptotic speedups over classical algorithms for certain tasks, and several of these tasks correspond closely to the core operations of sketch-based similarity analysis. First, similarity estimates obtained from sketches are inherently statistical: both sketch-based similarity measures and sketch-to-query containment scores can be expressed as probabilities of matching events. Classical approaches estimate such quantities by averaging independent random samples, which requires $O(1/\varepsilon^2)$ samples to reach additive error ε. Quantum amplitude estimation (QAE) [6] instead repeats the sampling procedure coherently and uses interference to encode the underlying proportion in a phase, which can be read out to the same additive error using $O(1/\varepsilon)$ repetitions. Second, search and minimum-selection operations are widely used in sketch-based similarity analysis. Grover's algorithm [13] uses amplitude amplification to provide a quadratic speedup for unstructured search, and related techniques can be extended to minimum finding [10] and, more generally, to finding the k smallest elements [9]. These observations indicate that quantum algorithms may be used to accelerate multiple stages of the sketching pipeline, from similarity estimation and large-scale screening to sketch construction. Realizing these speedups requires coherent access to the input data, so we adopt the quantum random access memory (QRAM) model [12], which allows stored data to be queried in superposition. Although its physical realization remains an open challenge, QRAM is a standard access model in the design and analysis of quantum algorithms [3]; it makes data-access costs explicit, and we account separately for the classical preprocessing needed to load the data.

Previous quantum approaches to set similarity have explored circuit constructions for matching and counting. Balewski et al. [2] developed quantum content-addressable memory (QCAM), which uses Grover search with optimized data-encoding circuits to find matching k-mers between two sequences, together with a hardware-efficient quantum counting procedure for choosing the number of Grover iterations. Their Jaccard computation recovers the shared k-mers by repeated sampling and counts them classically, so the number of measurements grows with the size of the intersection rather than with the target precision. Puram et al. [28] proposed reversible circuits that count the intersection and union of two binary vectors and compute their ratio classically. These circuits act

only on computational basis states, so they do not exploit superposition and their cost grows with vector length.

More recently, Kulkarni and Alam [17] introduced QuSim-Join, which combines classical locality-sensitive hashing with QAE-based verification of candidate pairs for set similarity joins, reducing the number of oracle calls per pair quadratically in the precision, and noted that containment admits an analogous encoding. However, the encoding circuit for each pair is built from the union and intersection of the two sets, which are computed classically beforehand. This computation already takes time linear in the set sizes and yields the Jaccard index exactly, and because it must be repeated for every pair, its cost cannot be amortized across comparisons. Our work builds on the same QAE precision improvement but preprocesses each set once into a pair-independent representation stored in QRAM, which can be queried coherently in polylogarithmic time. Preprocessing is thus amortized across comparisons, and the estimator can be used inside quantum search to combine its speedup from QAE with a speedup in the number of candidates examined.

In this work, we develop quantum algorithms for three stages of sequence comparison: pairwise similarity estimation, database search, and sketch construction. First, we reduce estimating the containment index of one k-mer set in another to estimating the success probability of a quantum procedure. After expected $\widetilde{O}(L)$ preprocessing time of two sequences of length at most L, quantum amplitude estimation gives an additive-ε estimate in $\widetilde{O}(1/\varepsilon)$ time, compared with $O(1/\varepsilon^2)$ membership queries for classical sampling, and the Jaccard index follows at the same asymptotic cost. Second, we use this estimator inside an error-tolerant quantum maximum-finding procedure to search a preprocessed database of R references. We find a reference whose containment index in a query is within ε of the best match in $\widetilde{O}(\sqrt{R}/\varepsilon)$ time, and a pair whose Jaccard index is within ε of the best pair in $\widetilde{O}(R/\varepsilon)$ time, compared with $\widetilde{O}(R/\varepsilon^2)$ and $\widetilde{O}(R^2/\varepsilon^2)$ for classical sampling. For best-pair search, the improvement persists even when preprocessing is included, provided that R^2/ε^2 dominates the total sequence length. Both searches can also run on sketches, which reduce the cost that scales with sequence length. Finally, we use quantum minimum finding to construct MinHash signatures for s prescribed hash functions in $\widetilde{O}(L + s\sqrt{n})$ time including preprocessing, where n is the number of distinct k-mers, compared with $\widetilde{O}(L+ns)$ for direct classical construction. We implement the core procedures in Q# and test them in small-scale simulation.

2 QAE-based containment index estimation

In this section, we present a QAE-based algorithm for estimating the containment index between the distinct k-mer sets of two sequences. We describe a QRAM implementation and prove its speedup over classical sampling after preprocessing. The estimator also serves as a subroutine for the database search algorithms developed in Section 3.

2.1 Problem definition

Let X and Y be two input sequences, and let A and B denote the sets of distinct k-mers occurring in X and Y, respectively. We assume that A is nonempty and let $a = |A|$, $b = |B|$, and $c = |A \cap B|$. The containment index of A in B is defined as $C(A \mid B) = \frac{|A \cap B|}{|A|} = \frac{c}{a}$.

Given A, B, and an additive accuracy parameter $\varepsilon > 0$, our goal is to output an estimate $\widetilde{C}(A \mid B)$ such that $\left|\widetilde{C}(A \mid B) - C(A \mid B)\right| \leq \varepsilon$ with constant success probability.

2.2 State preparation and membership oracle

We first prepare a uniform superposition over the distinct k-mers in the source set A. The state is $|\psi_A\rangle = \frac{1}{\sqrt{a}} \sum_{x \in A} |x\rangle$.

We then coherently determine whether each k-mer in the superposition also belongs to the target set B. To do so, we define a membership oracle for B that flips an ancillary qubit if and only if the queried k-mer is contained in B: $O_B |x\rangle |z\rangle = |x\rangle |z \oplus \mathbf{1}[x \in B]\rangle$, Applying this oracle to $|\psi_A\rangle$ with the ancillary qubit initialized to $|0\rangle$ gives $O_B (|\psi_A\rangle |0\rangle) = \frac{1}{\sqrt{a}} \sum_{x \in A} |x\rangle |\mathbf{1}[x \in B]\rangle$.

Since every element of A has the same squared amplitude $1/a$, the probability of observing the ancillary qubit in $|1\rangle$ is $p = \frac{1}{a} \sum_{x \in A} \mathbf{1}[x \in B] = \frac{|A \cap B|}{|A|} = C(A \mid B)$.

Thus, estimating the containment index reduces to estimating the probability p, which we do by QAE; Theorem 1 states the resulting guarantee. Figure 1 illustrates the construction on a small example.

Theorem 1 (Containment index estimation via QAE)**.** *Let A and B be finite sets with $A \neq \varnothing$, and let $0 < \varepsilon, \delta < 1$. Suppose we can implement a state-preparation oracle U_A that prepares the uniform state $|\psi_A\rangle$ over A, and the membership oracle O_B defined above. Then an additive-ε estimate of $C(A \mid B)$ with failure probability at most δ can be obtained using $O\left(\varepsilon^{-1} \log(2/\delta)\right)$ calls to the state-preparation and membership oracles.*

Including preprocessing, the total running time is $T_{\mathrm{pre}}(A, B) + \widetilde{O}\left(\frac{T_{\mathrm{prep}}(A) + T_{\mathrm{mem}}(B)}{\varepsilon} \log \frac{2}{\delta}\right)$, where $T_{\mathrm{pre}}(A, B)$ denotes the one-time cost of constructing the data representations used by the oracles, and $T_{\mathrm{prep}}(A)$ and $T_{\mathrm{mem}}(B)$ denote the costs of state preparation and membership queries, respectively.

Proof. The probability of the membership flag being $|1\rangle$ is $p = C(A \mid B)$. Standard QAE [6, Theorem 12] produces an estimate $\widetilde{p}$ satisfying $|\widetilde{p} - p| \leq \frac{2\pi\sqrt{p(1-p)}}{M} + \frac{\pi^2}{M^2} \leq \frac{\pi}{M} + \frac{\pi^2}{M^2}$ with probability at least $8/\pi^2 > 2/3$, using $O(M)$ state-preparation and membership calls. Choosing $M = \Theta(1/\varepsilon)$ with a sufficiently large constant makes the error at most ε. We output $\widetilde{C}(A \mid B) = \widetilde{p}$.

To ensure a failure probability of at most δ, we repeat the procedure independently $m = O(\log(2/\delta))$ times and return the median estimate. Since each run succeeds with probability at least $8/\pi^2 > 1/2$, the median lies within ε of $C(A \mid B)$ unless at least half of the runs fail. By a Chernoff bound, this event happens with probability at most δ for a suitable constant in m.

The data representations used by U_A and O_B are built once, at cost $T_{\mathrm{pre}}(A, B)$, and are reused by all $O\left(\varepsilon^{-1} \log(2/\delta)\right)$ oracle calls. Each call costs $\widetilde{O}(T_{\mathrm{prep}}(A) + T_{\mathrm{mem}}(B))$. This gives the stated bounds. □

2.3 QRAM implementation and preprocessing

We now implement the oracles of Theorem 1, keeping the notation of Section 2.1 and additionally writing $n = \max\{2, a, b\}$ and $L = \max\{|X|, |Y|\}$.

2.3.1 Access and cost assumptions

The QRAM interface allows memory locations to be addressed in quantum superposition [12]. We explicitly adopt the following cost model.

Assumption 1 (QRAM access and cost model)**.** For a classically stored array D of $N = O(a + b)$ words, QRAM supports the reversible indexed read $|i\rangle |z\rangle \mapsto |i\rangle |z \oplus D[i]\rangle$ on arbitrary superpositions of valid addresses in $\mathrm{polylog}(N)$ time.

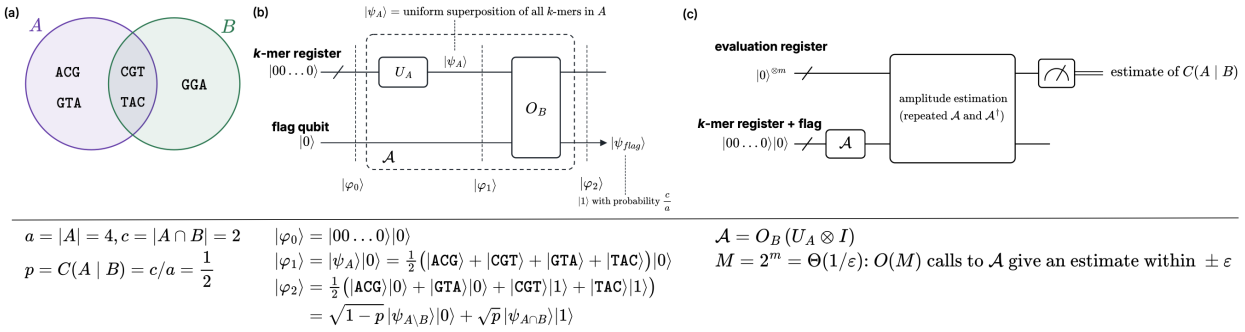


Figure 1: **QAE-based containment estimation.** (a) Example k-mer sets with containment index $p = C(A \mid B)$. (b) The operator $\mathcal{A}$ (dashed box) prepares the uniform superposition over the k-mers of A and flips a flag qubit for k-mers in B, so the flag is measured as $|1\rangle$ with probability p. Here $|\psi_S\rangle$ denotes the uniform superposition over a set S, and a slashed wire denotes a multi-qubit register. (c) Amplitude estimation uses $O(1/\varepsilon)$ calls to $\mathcal{A}$ and $\mathcal{A}^\dagger$ to estimate p within $\pm\varepsilon$ with constant probability.

The QRAM can be initialized and loaded with N memory words in $\widetilde{O}(N)$ time.

2.3.2 Preprocessing into hash-based data structures

We use hash-based data structures to store the k-mers of A and B, so that they can be looked up by content in space linear in the number of distinct k-mers, rather than in the 4^k entries for all possible k-mers. The distinct k-mers of X and Y are obtained by a standard linear scan with hash-based deduplication in expected $O(L)$ time. The distinct k-mers of A are also stored in an array $D_A[0], \ldots, D_A[a-1]$ in arbitrary order.

We also want lookups to be exact and to use a constant number of memory reads in the worst case, because both oracles query these data structures on superpositions of k-mers, so the circuit must accommodate the slowest lookup. We therefore build static cuckoo dictionaries [26] for A and B. For $S \in \{A, B\}$, the dictionary for S consists of two tables $H_{S,1}$ and $H_{S,2}$, each an array of r_S entries, and two hash functions $h_{S,1}$ and $h_{S,2}$ mapping k-mers to $\{0, \ldots, r_S - 1\}$; each $x \in S$ is stored at $H_{S,1}[h_{S,1}(x)]$ or $H_{S,2}[h_{S,2}(x)]$. Each entry holds an occupancy bit, the encoding of a k-mer, and, in the dictionary for A, the index of that k-mer in D_A. A lookup of x therefore reads exactly two entries and compares the stored k-mers with x, which meets both requirements.

Pagh and Rodler [26] show that if $r_S \geq (1+\gamma)|S|$ for a constant $\gamma > 0$ and the two hash functions are drawn independently from an $O(\log |S|)$-wise independent family, then all k-mers of S can be placed with probability $1 - O(1/|S|)$. We take $r_S = 2^{\lceil \log_2(2|S|) \rceil}$, so that $2|S| \leq r_S < 4|S|$ and each table is at most half full. For the hash family, we use random polynomials of degree $O(\log n)$ over a prime field; each has a description of $O(\log n)$ words and can be evaluated reversibly in polylog(n) time. We insert the k-mers with the standard cuckoo insertion procedure, verify that each is found in one of its two entries, and resample both hash functions if insertion fails. The expected number of attempts is $O(1)$, so both dictionaries are built in expected $\widetilde{O}(a+b)$ time; once verified, they remain fixed throughout estimation.

The array of k-mers, both dictionaries, and the hash-function descriptions occupy $O(a+b)$ QRAM words, which by Assumption 1 can be loaded in $\widetilde{O}(a+b)$ time. Since $a + b \leq 2L$, the expected preprocessing time is $\mathbb{E}[T_{\text{pre}}(A, B)] = O(L) + \widetilde{O}(a+b) = \widetilde{O}(L)$.

Standard hash tables based on chaining or linear probing achieve constant lookup time only in expectation. For example, even with a fully random hash function, chaining n k-mers into $\Theta(n)$

buckets produces a longest chain of length $\Theta(\log n/\log\log n)$ with probability $1-o(1)$ [20], and a reversible lookup would have to be unrolled to this worst case. Other static dictionaries with worst-case constant lookups, such as FKS perfect hashing [11], would also suffice.

2.3.3 Oracle implementation

Both the state-preparation and the membership oracles rely on the same reversible lookup. For $S \in \{A, B\}$ and a k-mer x held in a register, a lookup in the dictionary for S evaluates $h_{S,1}(x)$ and $h_{S,2}(x)$, reads the two slots through QRAM into temporary registers, and compares the stored k-mers with x. Since the lookup does not modify the register holding x, it can be uncomputed once its result has been used.

For the state-preparation oracle, we first prepare $\frac{1}{\sqrt{a}}\sum_{i=0}^{a-1}|i\rangle\,|D_A[i]\rangle$ by creating a uniform superposition over the indices $\{0,\dots,a-1\}$ with standard controlled rotations and performing one QRAM read of D_A. We then look up $D_A[i]$ in the dictionary for A, XOR the index i stored in the matching entry into the index register, and uncompute the lookup, which yields $|0\rangle\,|\psi_A\rangle$.

For the membership oracle, we look up the queried k-mer x in the dictionary for B, XOR into the flag whether either entry is occupied and stores x, and uncompute the lookup. This implements $O_B\,|x\rangle\,|z\rangle = |x\rangle\,|z \oplus \mathbf{1}[x \in B]\rangle$.

Each oracle uses $O(1)$ QRAM reads, and its remaining operations, namely preparing the index superposition, evaluating hash functions, comparing k-mers, and XORing registers, cost $\mathrm{polylog}(n)$. By Assumption 1, $T_{\mathrm{prep}}(A), T_{\mathrm{mem}}(B) \in \mathrm{polylog}(n)$.

2.3.4 Resulting estimation complexity

Corollary 1 (Containment estimation with QRAM preprocessing)**.** *In the setting of Section 2.3, let $0 < \varepsilon, \delta < 1$.*

Under Assumption 1, the construction above uses $O(a+b)$ QRAM words and requires expected $\widetilde{O}(L)$ preprocessing time. After preprocessing, an additive-ε estimate of $C(A \mid B)$ with failure probability at most δ can be obtained in time $\widetilde{O}\left(\varepsilon^{-1}\log(2/\delta)\right)$. Including preprocessing, the expected end-to-end time is $\widetilde{O}\left(L + \varepsilon^{-1}\log(2/\delta)\right)$.

Proof. Both state preparation and exact membership use $O(1)$ QRAM reads, including uncomputation, together with $\mathrm{polylog}(n)$ reversible computation. Consequently, Assumption 1 gives $T_{\mathrm{prep}}(A), T_{\mathrm{mem}}(B) \in \mathrm{polylog}(n)$, including the required inverse and controlled operations.

Applying Theorem 1 with these costs yields $\widetilde{O}\left(\varepsilon^{-1}\log(2/\delta)\right)$ estimation time after preprocessing. Adding the expected $\widetilde{O}(L)$ preprocessing cost establishes the end-to-end bound. The QRAM space bound follows from the compact array and the linear-size dictionaries and their hash-function descriptions. □

For constant failure probability, the estimation time after preprocessing is $\widetilde{O}(1/\varepsilon)$, and the expected end-to-end time is $\widetilde{O}(L + 1/\varepsilon)$.

Thus, once the sequences have been preprocessed, estimating their containment index takes time independent of the sequence lengths, up to polylog factors. For a single pair, the $O(L)$ preprocessing cost dominates, and an exact classical computation also takes expected $O(L)$ time. However, in a database setting each sequence can be preprocessed once and reused across many queries. Section 3 builds on this observation.

2.4 From containment to Jaccard index

The estimator also yields the Jaccard index $J(A,B) = |A \cap B|/|A \cup B|$. Since $|A \cap B| = a\, C(A \mid B)$,

$$J(A,B) = f\big(C(A \mid B)\big), \qquad f(p) = \frac{ap}{a+b-ap}.$$

Because $|A \cap B| \le b$, the containment lies in $I = [0,u]$ with $u = \min\{1, b/a\}$. The derivative $f'(p) = a(a+b)/(a+b-ap)^2$ is increasing, and $f'(u)$ equals $a(a+b)/b^2 \le 2$ if $a \le b$ and $(a+b)/a < 2$ otherwise. Thus f is 2-Lipschitz on I, so clipping an additive-η estimate of $C(A \mid B)$ to I and applying f gives an additive-2η estimate of $J(A,B)$. Running Theorem 1 with accuracy $\varepsilon/2$ therefore estimates $J(A,B)$ to additive accuracy ε at the same asymptotic cost.

3 Quantum search over a sequence database

The containment estimator developed in the previous section can be used as a score-evaluation subroutine for search problems in sequence databases. We consider two computation-heavy problems here.

3.1 Problem definition

Let $\mathcal{D} = (X_1, \dots, X_R)$ be a database of R reference sequences, and let $A_i \neq \varnothing$ denote the set of distinct k-mers of X_i. Let $0 < \varepsilon < 1$ be an additive accuracy parameter. We consider the following two problems, each to be solved with constant success probability.

1. **Query-to-database containment search.** Given a query sequence Q with distinct k-mer set A_Q, find an index $\widehat{i}$ whose containment index $C(A_{\widehat{i}} \mid A_Q)$ is within ε of $\max_i C(A_i \mid A_Q)$.

2. **Best-pair Jaccard search.** For $R \ge 2$, find a pair $(\widehat{i}, \widehat{j})$ whose Jaccard index $J(A_{\widehat{i}}, A_{\widehat{j}})$ is within ε of $\max_{i<j} J(A_i, A_j)$.

3.2 Query-to-database search

We first solve Problem 1 by running the containment estimator of Theorem 1 in superposition over the references inside a threshold-based maximum-finding procedure. This requires preparing $|\psi_{A_i}\rangle$ conditioned on a reference register $|i\rangle$, and we let $T^{\mathrm{DB}}_{\mathrm{prep}}$ denote the maximum cost of this controlled state preparation over all references.

We preprocess the database once by applying the cuckoo-dictionary construction of Section 2.3 to every reference: for each X_i, we extract A_i, store it in an array D_{A_i}, and build its cuckoo dictionary. All arrays and dictionaries are stored in a single QRAM, together with a table giving, for each i, the size $|A_i|$, the offsets of D_{A_i} and of its dictionary, and its hash-function description. The QRAM Assumption 1 is then applied with $N = O\big(\sum_i |A_i| + |A_Q|\big)$. Conditioned on $|i\rangle$, state preparation reads these entries into work registers, runs the state-preparation oracle of Section 2.3 on the corresponding array and dictionary, and then uncomputes the work registers, which depend only on i. Because $|A_i|$ is now held in a register, the rotation angles for the uniform superposition over $\{0, \dots, |A_i| - 1\}$ are computed coherently to $O(\log(1/\varepsilon))$ bits, which costs polylog time and perturbs the containment probability by at most a small constant fraction of ε. A query Q requires only a cuckoo dictionary for A_Q, since it is used only for membership queries. Under the QRAM Assumption 1, $T^{\mathrm{DB}}_{\mathrm{prep}}$ and $T_{\mathrm{mem}}(A_Q)$ are therefore polylogarithmic, the database is preprocessed once in expected $\widetilde{O}\big(\sum_i |X_i|\big)$ time, and each query is preprocessed in expected $\widetilde{O}(|Q|)$ time.

Let $\eta = \varepsilon/6$. Controlled on $|i\rangle$, we compute an estimate $\widetilde{C}_i$ of $C(A_i \mid A_Q)$ into a register using QAE with accuracy η, and take the median of $O(\log(R/\varepsilon))$ independent runs, so that $|\widetilde{C}_i - C(A_i \mid A_Q)| \le \eta$ except with probability $\delta' = 1/\operatorname{poly}(R/\varepsilon)$. This score estimator costs $\widetilde{O}\big((T_{\mathrm{prep}}^{\mathrm{DB}} + T_{\mathrm{mem}}(A_Q))/\varepsilon\big)$.

The search follows Dürr–Høyer maximum finding [10], but compares estimates against a classical threshold with a margin. Starting from an arbitrary index, it maintains a current index y and a threshold t, obtained by estimating the score of y and measuring. Let $\mathcal{A}$ prepare a uniform superposition over $\{1, \dots, R\}$, compute $\widetilde{C}_i$, and set a flag if $\widetilde{C}_i > t + 4\eta$. Each round applies amplitude amplification to $\mathcal{A}$, with the flag as the good subspace, using the BBHT schedule [4] with a budget of $O(\sqrt{R})$ applications. If a round measures a flagged index, that index becomes the new current index; after $\kappa = O(\log \log R)$ consecutive unsuccessful rounds, the search outputs y.

The margin serves two purposes. Every flagged index has a true score more than 2η above that of y, so every update moves to a strictly better candidate; and every index whose score exceeds $t + 5\eta$ is flagged, so the search stops only when the current candidate is close to the optimum.

Theorem 2 (Quantum query-to-database containment search)**.** *Consider Problem 1, and suppose that, conditioned on $|i\rangle$, the state $|\psi_{A_i}\rangle$ can be prepared in time $T_{\mathrm{prep}}^{\mathrm{DB}}$ and membership in A_Q can be queried in time $T_{\mathrm{mem}}(A_Q)$, including the inverse and controlled versions of both operations. Then, with constant success probability, the search above returns an index $\widehat{i}$ satisfying $C(A_{\widehat{i}} \mid A_Q) \ge \max_i C(A_i \mid A_Q) - \varepsilon$ in expected time $\widetilde{O}\left(\frac{\sqrt{R}\left(T_{\mathrm{prep}}^{\mathrm{DB}} + T_{\mathrm{mem}}(A_Q)\right)}{\varepsilon}\right)$. Under Assumption 1, with the preprocessing above, the search takes $\widetilde{O}(\sqrt{R}/\varepsilon)$ time, so a query costs $\widetilde{O}\big(|Q| + \sqrt{R}/\varepsilon\big)$ including its own preprocessing.*

Proof. Write $C_i = C(A_i \mid A_Q)$. By Theorem 1, each application of $\mathcal{A}$, including the median over $O(\log(R/\varepsilon))$ runs, costs $\widetilde{O}\big((T_{\mathrm{prep}}^{\mathrm{DB}} + T_{\mathrm{mem}}(A_Q))/\varepsilon\big)$. Replacing each estimator by one whose outputs are always η-accurate changes the state by $O(\sqrt{\delta'})$ in norm per application. Since the search uses $\widetilde{O}(\sqrt{R})$ applications in expectation, this changes its output distribution by $o(1)$, so we assume from now on that all estimates are η-accurate; in particular, $|t - C_y| \le \eta$.

Correctness. Let $G = \{x : C_x > t + 5\eta\}$ and $W = \{x : t + 3\eta < C_x \le t + 5\eta\}$. Every index in G is flagged, no index outside $G \cup W$ is flagged, and indices in W may be flagged with arbitrary probability. Hence a measured flagged index is uniform on G conditioned on lying in G. If $G \neq \varnothing$, the flagged probability mass is at least $1/R$, so each round succeeds with constant probability, and over the $O(\log R)$ expected updates bounded below, the search stops prematurely with probability at most a small constant. When the search stops with $G = \varnothing$, we have $\max_i C_i \le t + 5\eta \le C_y + 6\eta = C_y + \varepsilon$.

Number of updates. Let $n_y = |\{x : C_x > C_y + 2\eta\}|$. Since $t + 3\eta \ge C_y + 2\eta$, both G and W lie in this set, so $n_y \ge |G| + |W|$. If the new index x lies in G, every index above x lies in G, so n_x is at most the number of elements of G above x, whose expectation is at most $|G|/2$ by uniformity. If $x \in W$, then $C_x + 2\eta > t + 5\eta$, so $n_x \le |G|$. Writing $w \le |W|$ for the flagged mass in W, we obtain

$$\mathbb{E}[n_x] \le \frac{|G|}{|G| + w} \cdot \frac{|G|}{2} + \frac{w}{|G| + w} \cdot |G| \le \frac{|G| + w}{2} \le \frac{n_y}{2},$$

where the middle inequality follows from $|G|\big(|G|/2 + w\big) \le (|G| + w)^2/2$. Since $n_y \le R$ initially, and $G \cup W = \varnothing$ once $n_y = 0$, so that no index is flagged, the expected number of updates is $O(\log R)$.

Cost. The search uses at most κ rounds per update plus κ final rounds, hence $O(\kappa \log R)$ rounds in expectation, each with $O(\sqrt{R})$ applications of $\mathcal{A}$, together with one estimate per update to obtain the threshold. Multiplying $\widetilde{O}(\sqrt{R})$ applications by the cost of each gives the stated bound. □

After the same preprocessing, a classical algorithm that evaluates every reference performs R containment comparisons. Computing them exactly takes $O\big(\sum_i |A_i|\big)$ time, and estimating each to additive error ε by independent sampling takes $O(1/\varepsilon^2)$ membership queries per reference, or $\widetilde{O}(R/\varepsilon^2)$ in total. In contrast, QAE reduces the precision dependence from $1/\varepsilon^2$ to $1/\varepsilon$, and maximum finding reduces the number of candidates examined from R to $\widetilde{O}(\sqrt{R})$, giving $\widetilde{O}(\sqrt{R}/\varepsilon)$ under Assumption 1.

3.3 Best-pair search

Problem 2 has $\binom{R}{2} = \Theta(R^2)$ candidate pairs. Song and Kingsford [29] search such candidates with Dürr–Høyer maximum finding over the pair-index space $\{1, \dots, R\}^2$, checking the legality condition $i < j$ inside the marking circuit, but their analysis assumes a consistent comparison order. We keep their pair-index search structure and replace its comparison oracle by the margin-based threshold search of Section 3.2, which removes this assumption.

Every sequence is preprocessed as a reference in Section 3.2; its cuckoo dictionary serves both for state preparation, when the sequence is the source, and for membership queries, when it is the target. Conditioned on $|i, j\rangle$, we prepare $|\psi_{A_i}\rangle$, query membership in A_j, estimate $C(A_i \mid A_j)$ by QAE, and convert the estimate into an estimate of $J(A_i, A_j)$ by Section 2.4, using the sizes $|A_i|$ and $|A_j|$ read from QRAM. With containment accuracy $\eta/2$ and the median of $O(\log(R/\varepsilon))$ runs, the Jaccard estimate is within η of $J(A_i, A_j)$ except with probability $1/\operatorname{poly}(R/\varepsilon)$, at cost $\widetilde{O}(1/\varepsilon)$.

The search is then identical to that of Section 3.2, except that $\mathcal{A}$ prepares a uniform superposition over $\{1, \dots, R\}^2$ and flags a pair only if it is legal and its estimate exceeds $t + 4\eta$, and each round has a budget of $O(R)$ applications.

Corollary 2 (Quantum best-pair search)**.** *Under Assumption 1, after preprocessing all sequences in expected $\widetilde{O}\big(\sum_i |X_i|\big)$ time, a pair $(\widehat{i}, \widehat{j})$ with $\widehat{i} < \widehat{j}$ and $J(A_{\widehat{i}}, A_{\widehat{j}}) \geq \max_{i<j} J(A_i, A_j) - \varepsilon$ can be found with constant success probability in expected time $\widetilde{O}(R/\varepsilon)$. Including preprocessing, the expected end-to-end time is $\widetilde{O}\big(\sum_i |X_i| + R/\varepsilon\big)$.*

Proof. The proof of Theorem 2 uses only that the estimates are η-accurate except with probability $1/\operatorname{poly}(R/\varepsilon)$ and that $\mathcal{A}$ assigns equal amplitude to all candidates. Both hold here: illegal pairs are never flagged, and every legal pair has amplitude $1/R$. With $\binom{R}{2}$ candidates, the flagged probability mass is at least $1/R^2$ whenever some legal pair has score above $t + 5\eta$, each round uses $O(R)$ applications of $\mathcal{A}$, and the expected number of updates is $O(\log R)$. Since each application costs $\widetilde{O}(1/\varepsilon)$, the stated search time follows, and adding the preprocessing cost gives the end-to-end bound. □

Unlike the squared-Bhattacharyya search of [29], which gives speedup under a consistent-comparison assumption, this result requires no such assumption.

Exhaustive classical evaluation, by comparison, considers all $\binom{R}{2} = \Theta(R^2)$ pairs, and estimating each Jaccard index by sampling takes $O(1/\varepsilon^2)$ membership queries per pair. Including preprocessing, classical sampling takes $\widetilde{O}\big(\sum_i |X_i| + R^2/\varepsilon^2\big)$ time and the quantum search takes $\widetilde{O}\big(\sum_i |X_i| + R/\varepsilon\big)$, so the quantum search is asymptotically faster end to end whenever R^2/ε^2 dominates $\sum_i |X_i|$, for example once $R \gg \varepsilon^2 L$ when every sequence has length at most L.

3.4 Combination with sketching

Our method can be combined with sketching to obtain speedups along different dimensions. A FracMinHash sketch [15], for example, replaces each k-mer set by the subset of k-mers whose hash

values fall below a fixed fraction of the hash range, and the containment between sketches estimates the containment between the original sets [14]. Sketching reduces the size of each set, and hence the preprocessing time and QRAM space, while our method reduces the number of candidates examined from R to $\widetilde{O}(\sqrt{R})$ and the precision dependence from $1/\varepsilon^2$ to $1/\varepsilon$.

4 Quantum sketch construction

We now consider constructing reusable classical sketches. Our main focus is MinHash with multiple prescribed hash functions: one quantum-accessible input representation is reused to find all s minima.

4.1 Problem definition

Let X be an input sequence of length L, and let $A = \{x_1, \ldots, x_n\}$ be its nonempty set of distinct k-mers. Let $H = (h_1, \ldots, h_s)$ be a given collection of s independently seeded hash functions with exactly representable, ordered outputs. The corresponding MinHash signature is the classical vector $M_H(A) = (m_1(A), \ldots, m_s(A))$, where $m_r(A) = \min_{x \in A} h_r(x)$. Given X, H, and a failure parameter $\delta \in (0, 1/2)$, the task is to output every coordinate of $M_H(A)$ exactly with probability at least $1 - \delta$ over the algorithm's internal randomness and measurements.

The direct classical construction evaluates every h_r on every element of A, using $\Theta(ns)$ hash evaluations. Using the k-mer extraction of Section 2.3, constructing the signature from the raw sequence takes $\widetilde{O}(L + ns)$ time.

Let $T_{\mathrm{pre}}(A)$ denote the cost of extracting the distinct k-mers of X and loading them into the array D_A of Section 2.3, which supports the coherent indexed read $|i\rangle\,|z\rangle \mapsto |i\rangle\,|z \oplus x_i\rangle$.

We also require a reversible evaluation circuit for each h_r and an efficient preparation of the uniform index state $|u_n\rangle = n^{-1/2} \sum_{i=1}^{n} |i\rangle$. Let T_{val} bound the time for a coherent value evaluation, its inverse, a hash-value comparison, and a reflection about $|u_n\rangle$. Under QRAM access and efficient reversible hash evaluation, T_{val} is polylogarithmic in n and in the description size of each h_r.

4.2 Method

Fix a hash function h_r. Loading x_i, evaluating h_r, and uncomputing the loaded k-mer gives a value oracle $V_r\,|i\rangle\,|0\rangle = |i\rangle\,|h_r(x_i)\rangle$. For a current candidate hash value v, we implement the Grover phase oracle $O_{r,v}\,|i\rangle = (-1)^{f_{r,v}(i)}\,|i\rangle$, where $f_{r,v}(i) = \mathbf{1}[h_r(x_i) < v]$. The implementation computes the hash, compares it with v, applies the conditional phase, and uncomputes all workspace. Each phase-oracle call therefore uses only a constant number of data and hash evaluations.

The Dürr–Høyer minimum-finding algorithm [10], which starts from a random candidate and repeatedly searches for a smaller hash value, finds the minimum using $O(\sqrt{n})$ value-oracle queries with constant success probability. Repeating it $O(\log(s/\delta))$ times and retaining the smallest returned hash value reduces the coordinate's failure probability to at most δ/s. One successful run suffices, since every returned value is the hash of an actual input element.

We perform this procedure separately for $r = 1, \ldots, s$ and write each minimum to the classical signature; changing r changes only the hash seed and the search predicate.

4.3 Complexity analysis

Theorem 3 (MinHash construction for multiple hash functions)**.** *Under the access model above, the complete signature $M_H(A)$ can be constructed with probability at least $1 - \delta$ using $O(s\sqrt{n}\log(s/\delta))$*

calls to the data-access and hash-evaluation oracles. Its total time, including preprocessing and classical output, is $T_{\text{multi}} = T_{\text{pre}}(A) + O(T_{\text{val}} s\sqrt{n}\log(s/\delta) + s)$. *In particular, under Assumption 1 and the expected* $\widetilde{O}(L)$ *preprocessing of Section 2.3, the expected end-to-end time from* X *is* $\widetilde{O}(L + s\sqrt{n})$.

Proof. Each coordinate uses $O(\sqrt{n}\log(s/\delta))$ value queries and fails with probability at most δ/s. Summing the query counts and applying a union bound gives the query bound and overall success probability. Multiplying by T_{val} and adding the shared preparation cost $T_{\text{pre}}(A)$ and s classical word writes gives the time bound. Substituting the preprocessing and access costs gives the final statement. □

On success, the signature equals that of the direct classical construction with the same hashes, preserving its statistical properties and subsequent classical use. The theorem applies standard quantum minimum finding; the end-to-end benefit comes from sharing one input representation across all s searches.

Linear data preparation can therefore coexist with an advantage for multiple hash functions, because the direct classical computation evaluates ns input–hash pairs. For example, if $L = \Theta(n)$ and $s = \Theta(\sqrt{n})$, direct classical construction takes $\Theta(n^{3/2})$ hash evaluations, whereas the quantum algorithm takes $\widetilde{O}(n)$ total time including loading.

5 Implementation and simulation

We implemented the core procedures in Q# and tested them using quantum simulation. Our Q# implementation is available at https://github.com/Kingsford-Group/raphael. The implementation includes coherent indexed access, exact membership queries, QAE-based similarity estimation, score-based quantum search, and quantum minimum finding for prescribed hash functions. Memory access was implemented through explicit reversible multiplexing and equality tests, providing a small-instance functional realization rather than a physical QRAM implementation. These small set sizes are necessitated by the fact that we are simulating quantum hardware on a classical machine.

For containment estimation, we tested seven set-pair instances with source cardinalities of three or four, covering empty and complete overlap, an empty target, and a non-power-of-two source size. Each instance was evaluated in 64 independently seeded runs.

For sketch construction, we tested input sizes $n \in \{4, 8, 16\}$ with three prescribed affine hash functions $h_r(x) = (a_r x + b_r) \bmod 32$, using $(a_r, b_r) \in \{(3, 1), (5, 2), (7, 4)\}$.

Table 1 summarizes all 768 runs. Both search tasks returned optimal candidates in every observed run, and all constructed MinHash signatures were exact. One containment run and one Jaccard run exceeded the additive-error threshold of 0.1. These results provide functional checks on the tested instances.

Task	Instance size	Metric	Mean	Successes
Containment QAE	$\lvert A\rvert \in \{3, 4\}$; 7 instances	Absolute error	0.0104	447/448
Jaccard via QAE	$\lvert A\rvert = \lvert B\rvert = 4$	Absolute error	0.0260	63/64
Query search	$R = 4$; $\lvert A_i\rvert = 4$	Score regret	0.0000	32/32
Best-pair search	$R = 4$; 6 legal pairs	Score regret	0.0000	32/32
Multi-hash MinHash	$n \in \{4, 8, 16\}$; $s = 3$	Wrong coordinates	0.0000	192/192

Table 1: Functional correctness in noiseless Q# simulations. Success requires absolute error or score regret at most 0.1, or an exactly correct complete MinHash signature.

6 Conclusion

We developed quantum algorithms for three stages of sequence comparison under a QRAM model. Quantum amplitude estimation estimates containment and Jaccard indices with quadratically better dependence on precision than classical sampling; error-tolerant maximum finding uses this estimator to search a database while examining quadratically fewer candidates; and quantum minimum finding constructs MinHash signatures for multiple hash functions with quadratically fewer hash evaluations in the number of distinct k-mers. Because each algorithm requires linear-time preprocessing, the total running time improves only when the preprocessed representation is reused across many estimates, searches, or hash functions.

Future work includes developing coherent-access implementations with lower preparation and memory costs, and determining which advantages continue to hold under weaker access assumptions, including models without QRAM. Resource estimation and evaluation on realistic genomic datasets should account for circuit depth, noise, data loading, and comparison with optimized classical pipelines. Other directions include quantum walks for database search and quantum construction of order-sensitive sketches related to edit distance.

References

[1] Alexander Astashyn, Eric S Tvedte, Deacon Sweeney, Victor Sapojnikov, Nathan Bouk, Victor Joukov, Eyal Mozes, Pooja K Strope, Pape M Sylla, Lukas Wagner, et al. Rapid and sensitive detection of genome contamination at scale with FCS-GX. *Genome Biology*, 25(1):60, 2024.

[2] Jan Balewski, Daan Camps, Katherine Klymko, and Andrew Tritt. Efficient quantum counting and quantum content-addressable memory for DNA similarity. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 378–384. IEEE, 2023.

[3] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549(7671):195–202, 2017.

[4] Michel Boyer, Gilles Brassard, Peter Høyer, and Alain Tapp. Tight bounds on quantum searching. *Fortschritte der Physik: Progress of Physics*, 46(4-5):493–505, 1998.

[5] Phelim Bradley, Henk C Den Bakker, Eduardo PC Rocha, Gil McVean, and Zamin Iqbal. Ultrafast search of all deposited bacterial and viral genomic data. *Nature Biotechnology*, 37(2):152–159, 2019.

[6] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. *arXiv preprint quant-ph/0005055*, 2000.

[7] Andrei Z Broder. On the resemblance and containment of documents. In *Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No. 97TB100171)*, pages 21–29. IEEE, 1997.

[8] Pierre-Alain Chaumeil, Aaron J Mussig, Philip Hugenholtz, and Donovan H Parks. GTDB-Tk: a toolkit to classify genomes with the Genome Taxonomy Database, 2020.

[9] Christoph Dürr, Mark Heiligman, Peter Høyer, and Mehdi Mhalla. Quantum query complexity of some graph problems. *SIAM Journal on Computing*, 35(6):1310–1328, 2006.

[10] Christoph Dürr and Peter Høyer. A quantum algorithm for finding the minimum. *arXiv preprint quant-ph/9607014*, 1996.

[11] Michael L Fredman, János Komlós, and Endre Szemerédi. Storing a sparse table with $O(1)$ worst case access time. *Journal of the ACM (JACM)*, 31(3):538–544, 1984.

[12] Vittorio Giovannetti, Seth Lloyd, and Lorenzo Maccone. Quantum random access memory. *Physical Review Letters*, 100(16):160501, 2008.

[13] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 212–219, 1996.

[14] Mahmudur Rahman Hera, N. Tessa Pierce-Ward, and David Koslicki. Deriving confidence intervals for mutation rates across a wide range of evolutionary distances using FracMinHash. *Genome Research*, 33(7):1061–1068, 2023.

[15] Luiz Irber, Phillip T Brooks, Taylor Reiter, N Tessa Pierce-Ward, Mahmudur Rahman Hera, David Koslicki, and C Titus Brown. Lightweight compositional analysis of metagenomes with FracMinHash and minimum metagenome covers. *BioRxiv*, pages 2022–01, 2022.

[16] Chirag Jain, Luis M Rodriguez-R, Adam M Phillippy, Konstantinos T Konstantinidis, and Srinivas Aluru. High throughput ANI analysis of 90K prokaryotic genomes reveals clear species boundaries. *Nature Communications*, 9(1):5114, 2018.

[17] Prateek P Kulkarni and Aakarsh Alam. QuSim-Join: Provably optimal quadratic speedup in set similarity joins via quantum amplitude estimation. In *Proceedings of the 3rd workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications*, pages 10–22, 2026.

[18] Guillaume Marçais, Dan DeBlasio, Prashant Pandey, and Carl Kingsford. Locality-sensitive hashing for the edit distance. *Bioinformatics*, 35(14):i127–i135, 2019.

[19] Guillaume Marçais, Brad Solomon, Rob Patro, and Carl Kingsford. Sketching and sublinear data structures in genomics. *Annual Review of Biomedical Data Science*, 2(1):93–118, 2019.

[20] Michael Mitzenmacher and Eli Upfal. Probability and computing. *Cambridge University*, 2005.

[21] Michael A Nielsen, Isaac L Chuang, et al. *Quantum computation and quantum information*, volume 1. Cambridge University Press, Cambridge, 2000.

[22] Nuala A O'Leary, Mathew W Wright, J Rodney Brister, Stacy Ciufo, Diana Haddad, Rich McVeigh, Bhanu Rajput, Barbara Robbertse, Brian Smith-White, Danso Ako-Adjei, et al. Reference sequence (RefSeq) database at NCBI: current status, taxonomic expansion, and functional annotation. *Nucleic Acids Research*, 44(D1):D733–D745, 2016.

[23] Matthew R Olm, Christopher T Brown, Brandon Brooks, and Jillian F Banfield. dRep: a tool for fast and accurate genomic comparisons that enables improved genome recovery from metagenomes through de-replication. *The ISME Journal*, 11(12):2864–2868, 2017.

[24] Brian D Ondov, Gabriel J Starrett, Anna Sappington, Aleksandra Kostic, Sergey Koren, Christopher B Buck, and Adam M Phillippy. Mash Screen: high-throughput sequence containment estimation for genome discovery. *Genome Biology*, 20(1):232, 2019.

[25] Brian D Ondov, Todd J Treangen, Páll Melsted, Adam B Mallonee, Nicholas H Bergman, Sergey Koren, and Adam M Phillippy. Mash: fast genome and metagenome distance estimation using MinHash. *Genome Biology*, 17(1):132, 2016.

[26] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. *Journal of Algorithms*, 51(2):122–144, 2004.

[27] Donovan H Parks, Maria Chuvochina, Christian Rinke, Aaron J Mussig, Pierre-Alain Chaumeil, and Philip Hugenholtz. GTDB: an ongoing census of bacterial and archaeal diversity through a phylogenetically consistent, rank normalized and complete genome-based taxonomy. *Nucleic Acids Research*, 50(D1):D785–D794, 2022.

[28] Varun Puram, Ruthvik Rao Bobbili, and Johnson P Thomas. Quantum algorithm for Jaccard similarity. *arXiv preprint arXiv:2408.08940*, 2024.

[29] Zhezheng Xander Song and Carl Kingsford. Quantum closest-pair search for biological sequences via k-mer distribution statistics. In *26th International Conference on Algorithms for Bioinformatics (WABI 2026)*, pages 9–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2026.

[30] Derrick E Wood, Jennifer Lu, and Ben Langmead. Improved metagenomic analysis with Kraken 2. *Genome Biology*, 20(1):257, 2019.